第 16 章：装饰器、闭包与工具注册

一句话理解本章：**装饰器是在执行 `def` 时，把刚创建的函数交给另一个可调用对象处理，再用处理结果替换原函数名。**

因此，装饰器既可以给每次调用增加计时、日志或鉴权，也可以只在定义函数时把它登记到注册表。闭包让包装函数在以后执行时仍能找到原函数和配置；`functools.wraps` 与 `ParamSpec` 则分别保住运行时元信息和静态类型关系。

学完本章后，你应该能够：

- 把 `@decorator` 手工展开为函数调用和名字重新绑定；
- 区分模块导入、装饰器应用和业务函数调用三个时刻；
- 解释闭包、`nonlocal`、跨调用共享状态和循环变量晚绑定；
- 写出能正确转发参数、返回值和异常的同步 wrapper；
- 判断 `wraps`、`ParamSpec` 各自解决了什么问题；
- 理解同步 `@tool(...)` 为什么能在导入时注册，却不会执行工具函数体。

本章只讨论**同步函数装饰器**。异步函数、生成器、重试、缓存和并发状态管理有不同的生命周期，不应塞进一个所谓的“万能装饰器”。

16.1 Java 工程师先建立什么心智模型

装饰器常被类比成 Java 注解或 Spring AOP。这个类比能帮助入门，但不能直接画等号。

Java 中熟悉的概念	Python 中可以先类比为	不能等同的地方
`@Transactional`、拦截器、动态代理	用 wrapper 增加横切行为	Java 注解本身通常只是元数据，由容器或代理解释；Python 装饰器表达式会真实执行，并直接改变名字绑定
带属性的 Java 注解	`@tool(name="search")` 装饰器工厂	Python 中 `tool(...)` 是普通函数调用，不是被动标签
Lambda / 匿名内部类捕获局部状态	闭包捕获外层名字	Java 有 effectively final 限制；Python 可以读取外层绑定，重新绑定时用 `nonlocal`

!502x6dg8fpjj0sff40000gp/T/crossnote-f-f695acc98bca1129d253380c8.html
eclp相关 配运运单26年需求... 运单小秘值班安排 研发日常任务 运单运维 运单-资源大屏 20260618-应急预案 运单26年需求迭代... 商家下单计数

Spring Bean 注册表	模块导入时把函数写入字典	这里没有类路径扫描、容器生命周期或依赖注入，只有 Python 顶层代码和进程内对象
泛型适配器保持函数接口	`ParamSpec` + 返回类型变量	它只帮助 Pyright 等静态检查器，不会在运行时验证实参
代理对象暴露目标方法信息	`functools.wraps` 和 `__wrapped__`	`wraps` 复制常用元信息并建立解包链，不会自动保证业务语义或静态类型正确

最关键的差异是：Java 注解通常要等框架读取后才产生效果，而 Python 的 `@...` 本身就是函数定义语句的一部分。解释器执行到它时，装饰逻辑已经发生。

16.2 `@decorator` 到底做了什么

函数名保存的是函数对象

Python 函数可以像普通对象一样赋给变量、作为参数传入，也可以作为返回值返回：

```
def greet(name: str) -> str:
    return f"你好，{name}"


operation = greet  # 没有括号：保存函数对象
message = operation("Ada")  # 有括号：执行函数体
```

这与 Java 中把方法引用保存为 `Function<String, String>` 的用途相似。不同之处是，Python 的函数对象还带有 `__name__`、`__doc__`、注解和签名等可内省信息。

装饰器就是“接收函数，返回替代对象”

先不用 `@`，写一个最小装饰器：

```
def announce_call(function):
    def wrapper():
        print("准备调用")
        return function()

    return wrapper
```

```
def message():
    return "完成"


message = announce_call(message)
```

最后一行发生了三件事：

1. 右侧 `message` 取得原函数对象；
2. `announce_call(message)` 创建并返回 `wrapper`；
3. 左侧 `message` 改为指向这个 wrapper。

以后调用 `message()`，实际先进入 wrapper，再由闭包中的 `function()` 调用原函数。

`@` 只是把这个关系写在函数定义旁边：

```
@announce_call
def message():
    return "完成"
```

理解装饰器时，始终先写出这条等式：

```
message = announce_call(message)
```

准确地说，真实语义不会先把原函数临时绑定给 `message` 再赋值，但这条等式能正确表达最终的对象关系。Python 语言参考也用这种“近似等价”方式解释函数装饰器语义。

装饰器的返回值不一定非得是函数，但它会成为公开名字的新值。如果装饰器意外返回 `None`，定义完成后 `message` 就是 `None`，稍后执行 `message()` 才会抛出 `TypeError`。

16.3 定义时和调用时是两条时间线

假设工具模块中有：

```
@tool(name="search", description="搜索文档")
def search(query: str) -> list[str]:
    return send_http_request(query)
```

不要把“注册工具”和“调用工具”理解成同一件事：

时刻	会发生什么	不会发生什么
首次导入模块	Python 从上到下执行模块顶层语句	不会自动调用所有函数体
到达带装饰器的 `def`	先从上到下求值装饰器表达式；`tool(...)` 返回真正的装饰器	`search` 的函数体仍未执行
准备并创建 `search` 函数对象	计算默认参数等定义期表达式，然后构造函数对象	`return` 和 HTTP 请求还没有执行
应用装饰器	把函数交给装饰器；多层时从下到上应用，最后绑定公开名字	仍未发送业务请求
以后调用 `search(...)`	wrapper 和原函数体开始执行	此时通常不再做首次注册

“定义时”不是保存源码的时刻，而是解释器真的执行这条 `def` 语句的时刻。模块级 `def` 通常在首次导入时执行；如果 `def` 位于另一个函数内部，每次控制流走到那里，都可能重新创建函数并再次应用装饰器。

普通导入会使用 `sys.modules` 缓存。同一进程再次 `import` 已加载模块，通常不会重跑顶层代码。但新进程、显式重载或用另一个模块名加载同一文件仍可能再次注册，所以注册表仍要主动拒绝重名。

工程上的安全边界也很清楚：模块导入时可以做轻量、确定性的声明与注册，但不要顺便建立远程连接、发 HTTP 请求或调用模型。导入应该能发现工具，不应该执行业务。

16.4 闭包为什么能记住原函数和配置

内层函数引用了外层函数的局部名字，外层返回后这些绑定仍可被内层函数访问，这就是闭包（closure）：

```
def make_prefixer(prefix: str):
    def add_prefix(text: str) -> str:
        return f"{prefix}: {text}"

    return add_prefix


warning = make_prefixer("WARN")
```

```
warning = make_prefixer("WARN")
print(warning("磁盘空间不足"))  # WARN: 磁盘空间不足
```

`make_prefixer()` 已经结束，但 `warning` 仍能读取那次调用中的 `prefix`。装饰器的 wrapper 也是同样的机制：它通常捕获原函数、装饰器配置以及需要跨调用共享的状态。

重新绑定外层名字时要用 `nonlocal`

下面的计数器让同一个 wrapper 的多次调用共享计数：

```
def count_calls(function):
    count = 0

    def wrapper(*args, **kwargs):
        nonlocal count
        count += 1
        print(f"第 {count} 次调用")
        return function(*args, **kwargs)

    return wrapper
```

整数不可变，`count += 1` 会计算新整数并把名字 `count` 重新绑定。没有 `nonlocal` 时，Python 会把 wrapper 中被赋值的 `count` 判定为局部变量；右侧先读取尚未赋值的局部变量，于是抛出 `UnboundLocalError`。

如果捕获的是列表，`events.append(...)` 只是修改同一个对象，不需要 `nonlocal`；如果写成 `events = [...]`，那又是重新绑定，仍需要 `nonlocal`。

每次应用装饰器都会创建一个新 wrapper 和一份新闭包。因此，一个被装饰函数的多次调用共享状态，两个分别被装饰的函数默认不共享同一个计数。这里的共享状态也不会自动拥有线程安全、请求隔离或清理机制，生产代码必须单独设计这些边界。

闭包捕获绑定，不会自动拍快照

循环中最常见的错误是晚绑定：

```
readers = []
for name in ["search", "weather", "database"]:
```

```
readers = []
for name in ["search", "weather", "database"]:

    def read_name():
        return name

    readers.append(read_name)

print([reader() for reader in readers])
# ['database', 'database', 'database']
```

循环不会为每一轮创建新的函数作用域。三个函数对象都在调用时读取同一个 `name` 绑定，而循环结束后它的值是 `"database"`。

更清楚的修复是用工厂函数为每个值创建独立作用域：

```
def make_reader(name: str):
    def read_name() -> str:
        return name

    return read_name


names = ["search", "weather", "database"]
readers = [make_reader(name) for name in names]
```

也可以用 `def read_name(name=name)` 把当前值放进默认参数，但这会把本不应公开的实现细节变成可传参数。小工厂通常更容易表达意图。

16.5 无参装饰器、装饰器工厂和多层顺序

无参装饰器直接接收函数：

```
@timed
def work(): ...


# 近似等价于
work = timed(work)
```

README
readme
README
README
wf7lz72502x6dg8fpjj0sff40000gp/T/crossnote-f-f695acc98bca1129d253380c8.html
相关...
eclp相关
配运运单26年需求...
运单小秘值班安排
研发日常任务
运单运维
运单-资源大屏
20260618-应急预案
运单26年需求迭代...
商家下单计数

16.5 无参装饰器、装饰器工厂和多层顺序

无参装饰器直接接收函数：

```
@timed
def work(): ...


# 近似等价于
work = timed(work)
```

带参形式先调用一个工厂，再由工厂返回的装饰器接收函数：

```
@tool(name="search", description="搜索文档")
def search(query: str): ...


# 近似等价于
search = tool(name="search", description="搜索文档")(search)
```

这里最多有三层调用，不要混在一起：

1. `tool(...)` 接收配置，在定义时返回 `decorate`；
2. `decorate(function)` 接收刚创建的函数，在定义时注册或包装；
3. `wrapper(*args, **kwargs)` 在以后每次业务调用时执行。

多个装饰器还要区分三种顺序：

```
@outer("A")
@inner
def work():
    print("body")
```

先展开成：

```
work = outer("A")(inner(work))
```

于是：

README | readme | README | README
/wf7lz72502x6dg8fpjj0sff40000gp/T/crossnote-f-f695acc98bca1129d253380c8.html
相关... | eclp相关 | 配运运单26年需求... | 运单小秘值班安排 | 研发日常任务 | 运单运维 | 运单-资源大屏 | 20260618-应急预案 | 运单26年需求迭代... | 商家下单计数

于是：

```
定义阶段：求值 outer("A") → 求值 inner → 应用 inner → 应用 outer
调用阶段：进入 outer wrapper → 进入 inner wrapper → 函数体 → 退出 inner → 退出 outer
```

最靠近函数的装饰器先应用，最终返回的最外层 wrapper 却最先接收调用。装饰器顺序不只影响日志，还会决定注册表保存哪个对象：

```python
@tool(name="sum")
@timed
def add(left: int, right: int) -> int:
    return left + right
```

它展开为 `add = tool(name="sum")(timed(add))`，所以注册表保存计时 wrapper，通过注册表调用也会计时。若交换顺序，`tool` 可能先保存原函数，注册表调用便会绕过后来添加的计时层。遇到多层装饰器，不要凭直觉猜，先展开括号。

16.6 怎样写一个尽量透明的同步 wrapper

所谓“透明”，是指增加横切行为后，原函数的参数规则、返回结果和异常语义仍然成立，并尽量保留原来的元信息与静态类型。

参数、返回值和异常必须原样通过

定义参数时，`*args` 把额外位置参数收集成元组，`**kwargs` 把关键字参数收集成字典；调用函数时，`*args` 和 `**kwargs` 又把它们展开：

```python
def wrapper(*args, **kwargs):
    return function(*args, **kwargs)
```

wrapper 表面上能接收任意参数，不代表原函数真的接受任意调用。转发后，仍由原函数检查仅位置参数 `/`、仅关键字参数 `*`、默认值、缺失参数和重复传值。

`return` 不能漏。下面的原函数虽然执行了，调用方却只能收到 `None`：

```python
def wrapper(*args, **kwargs):
    function(*args, **kwargs)  # 错误：丢掉原返回值
```

:72502x6dg8fpjj0sff40000gp/T/crossnote-f-f695acc98bca1129d253380c8.html

eclp相关 配运运单26年需求... 运单小秘值班安排 研发日常任务 运单运维 运单-资源大屏 20260618-应急预案 运单26年需求迭代... 商家下单计数

也不要为了打印和返回结果而调用两次 `function(...)`。对于写数据库、发请求或执行 tool 的函数，这会让副作用重复发生。正确做法是调用一次，把结果存起来再使用。

计时逻辑希望成功和失败都被记录，可以使用 `try/finally`：

```
started = clock()
try:
    return function(*args, **kwargs)
finally:
    report(clock() - started)
```

`finally` 执行后，正常返回值或原异常会继续向上传递。若随手写成 `except Exception: return None`，就把"调用失败"和"合法返回空值"混成了同一种结果，这已经改变原函数契约。

`wraps` 保留运行时元信息

包装后公开名字实际指向 wrapper。若不处理，日志、帮助系统和注册器看到的往往是 `wrapper`、空 docstring 和宽泛的 `*args, **kwargs` 签名。

```
from functools import wraps


def decorate(function):
    @wraps(function)
    def wrapper(*args, **kwargs):
        return function(*args, **kwargs)

    return wrapper
```

`@wraps(function)` 会复制常用元信息，并设置 `wrapper.__wrapped__ = function`：

检查方式	没有 `wraps`	使用 `wraps`
`decorated.__name__`	通常是 `wrapper`	原函数名
`decorated.__doc__`	wrapper 的说明或 `None`	原 docstring
`decorated.__wrapped__`	不存在	指向下一层被包装对象
`inspect.signature(decorated)`	常见为 `*args, **kwargs`	默认沿 `__wrapped__` 展示原签名

README | readme | README | README
lz72502x6dg8fpjj0sff40000gp/T/crossnote-f-f695acc98bca1129d253380c8.html
eclp相关 | 配运运单26年需求... | 运单小秘值班安排 | 研发日常任务 | 运单运维 | 运单-资源大屏 | 20260618-应急预案 | 运单26年需求迭代... | 商家下单计数

`inspect.signature(decorated, follow_wrapped=False)` 可以查看 wrapper 自身的入口。直接调用 `decorated.__wrapped__` 则会绕过计时、鉴权等包装行为，只应在明确需要调试、内省或重新包装时使用。

`wraps` 只处理运行时元信息，不会证明 wrapper 的类型注解正确。完整规则可查阅 `functools.wraps` 和 `inspect.signature`。

`ParamSpec` 保留静态调用关系

对于“接收任意函数并保持原签名”的装饰器，真正想表达的不是“什么都可以”，而是：包装前后必须接受同一组参数并返回同一种结果。

本项目使用 Python 3.14 类型参数语法：

```
from collections.abc import Callable
from functools import wraps


def timed[**P, R](function: Callable[P, R]) -> Callable[P, R]:
    @wraps(function)
    def wrapper(*args: P.args, **kwargs: P.kwargs) -> R:
        return function(*args, **kwargs)

    return wrapper
```

- `P` 是 `ParamSpec`，表示整组参数规格，不是某个参数值；
- `P.args` 和 `P.kwargs` 分别描述这组参数的位置与关键字部分；
- `R` 表示返回类型；
- 输入和输出都是 `Callable[P, R]`，因此 Pyright 能保留参数名、位置规则和返回值关系。

需要兼容 Python 3.11 及更早版本的项目，会看到模块级 `P = ParamSpec("P")` 和 `R = TypeVar("R")` 写法。两种形式表达的是同一个类型关系。

`ParamSpec` 服务于静态检查，不会在运行时验证参数；`wraps` 服务于运行时内省，不会建立静态类型关系。一个工程化 wrapper 通常两者都需要。详见 Python `ParamSpec` 文档。

一个完整的同步计时器

把控制流、元信息和类型关系合在一起：

一个完整的同步计时器

把控制流、元信息和类型关系合在一起：

```
from collections.abc import Callable
from functools import wraps
from time import perf_counter_ns


def timed[**P, R](function: Callable[P, R]) -> Callable[P, R]:
    @wraps(function)
    def wrapper(*args: P.args, **kwargs: P.kwargs) -> R:
        started_ns = perf_counter_ns()
        try:
            return function(*args, **kwargs)
        finally:
            elapsed_ms = (perf_counter_ns() - started_ns) / 1_000_000
            print(f"{function.__name__}: {elapsed_ms:.3f} ms")

    return wrapper
```

`perf_counter_ns()` 适合测量进程内时间间隔。具体毫秒值受机器负载影响，不应写进固定断言。这个实现只包围一次同步函数调用；如果传入协程函数，它测到的主要是“创建协程对象”的时间，而不是真正 `await` 完成的时间，因此不能当异步计时器使用。

16.7 装饰器也可以只注册、不包装

装饰器只要求接收函数并返回用于重新绑定的对象，并不要求一定创建 wrapper。最小注册器可以这样写：

```
registry = {}


def tool(*, name: str):
    def decorate(function):
        if name in registry:
            raise ValueError(f"工具名称已注册：{name}")
        registry[name] = function
        return function
```

这个 `@tool(...)` 在定义时完成注册，然后直接返回原函数，所以：

- 函数对象身份不变；
- 调用时没有额外 wrapper；
- 装饰器不能在每次调用前后插入行为；
- 注册表保存的函数与公开名字是同一个对象。

如果需要统一鉴权、验证或日志，就应创建 wrapper，并让公开名字与注册表都保存同一个 wrapper。代价是必须正确处理参数、结果、异常、元信息和类型关系。选择标准只有一个：**是否必须拦截每次调用。** 不需要拦截时，不要为了形式统一增加空 wrapper。

本章注册表保存什么

`tool_registry.py` 中的 `ToolMetadata` 保存：

字段	来源	用途
`name`	装饰器参数，缺省时用函数名	注册表查找和模型选择工具
`python_name`	`function.__name__`	日志和排错
`description`	显式参数，缺省时用 docstring	描述工具用途
`signature`	`inspect.signature()`	参数名、参数种类、默认值和返回注解
`annotations`	`annotationlib.get_annotations()`	生成参数描述或 schema 的原料
`function`	被注册的同步函数	真正执行调用

注册器先校验名称和说明，再检查重名，最后才写入字典。这样失败不会覆盖旧函数，也不会留下半注册状态。真实系统若允许替换，应设计显式的 `replace=True` 和来源记录，而不是依赖普通字典覆盖。

这个注册表仍不是完整的 Agent tool runtime：类型注解不会自动验证模型传入的 JSON，任意 Python 类型也不一定能转换成 JSON Schema。外部输入仍需要明确的 schema、反序列化和运行时校验。

动态注册会遇到静态类型边界

直接调用具体函数时，Pyright 知道它的精确签名：

```
results: list[str] = search_documents("装饰器", limit=2)
```

README
readme
README
README
/wf7lz72502x6dg8fpjj0sff40000gp/T/crossnote-f-f695acc98bca1129d253380c8.html
相关...
eclp相关
配运运单26年需求...
运单小秘值班安排
研发日常任务
运单运维
运单-资源大屏
20260618-应急预案
运单26年需求迭代...
商家下单计数

动态注册会遇到静态类型边界

直接调用具体函数时，Pyright 知道它的精确签名：

```
results: list[str] = search_documents("装饰器", limit=2)
```

但注册表把多种签名放进同一个字典，再通过运行时字符串选择：

```
result = invoke_tool("summarize_results", items, prefix="命中")
```

普通静态检查器无法根据任意 `str` 推导对应函数的参数和返回类型，因此本章的统一动态入口诚实地返回 `object`。不要用 `Any` 把检查关闭后，再声称类型关系仍然存在。动态边界应由运行时 schema 验证兜底。

16.8 Python 3.14 如何读取延迟注解

本项目要求 Python `>=3.14`。从 Python 3.14 开始，注解默认延迟求值：执行 `def` 时不必马上把所有注解解析成真实对象，访问注解时才求值。

这使下面的前向引用合法：

```
@tool(name="render_card", description="渲染结果")
def render_card(card: ResultCard) -> str: ...


class ResultCard: ...
```

应用 `@tool` 时，`ResultCard` 还没有定义。本章使用公开 API 明确选择所需格式：

```
from annotationlib import Format, get_annotations
from inspect import signature

annotations = get_annotations(function, format=Format.FORWARDREF)
display_signature = signature(function, annotation_format=Format.STRING)
```

格式	得到什么	适用场景
`Format.VALUE`	尽量求值得到真实对象，名字未定义时可能失败	模块加载完成后确实需要类型对象

格式	得到什么	适用场景
`Format.VALUE`	尽量求值得到真实对象，名字未定义时可能失败	模块加载完成后确实需要类型对象
`Format.FORWARDREF`	已知项为真实值，未知项为 `ForwardRef`	定义期注册并允许前向引用
`Format.STRING`	接近源码的文本表示	文档、日志和面向人的签名

在 `registered_tools.py` 中，`render_card` 注册时得到 `ForwardRef('ResultCard')`；模块完成导入后，再用 `Format.VALUE` 读取便能得到真正的 `ResultCard` 类。这说明“延迟求值”既不等于“永远是字符串”，也不等于“注解永远不会执行”。

不要让框架代码依赖 `__anhotations__` 在某个 Python 版本中的具体内部形式。Python 3.14 应优先使用 `annotationlib.get_annotations()`；需要递归解析复杂类型时，再评估 `typing.get_type_hints()`。

注解内省还有安全边界：求值注解表达式可能执行代码，`Format.STRING` 也不是加载不可信 Python 源码的沙箱。本章只导入并内省仓库内受信任的模块。更多细节见 `annotationlib` 文档。

16.9 Java 工程师最容易踩的坑

错误判断	直接后果	正确做法
把 `@tool` 当成不执行的 Java 元数据	看不懂为何导入时已注册	记住装饰器表达式和装饰器都会在执行 `def` 时调用
wrapper 少写 `return`	原函数可能已产生副作用， 但调用方只收到 `None`	`return function(*args, **kwargs)`
忘记 `wraps`	名字、docstring、 签名和注册元信息属于 wrapper	在 wrapper 上使用 `@wraps(function)`
用 `Callable[..., Any]` 宣称类型已保留	错参数和错返回值更容易漏检	用 `ParamSpec` 与返回类型变量建立对应关系
宽泛捕获异常后返回 `None`	失败与合法空值无法区分	默认传播异常；若要转换， 明确设计新契约
wrapper 调用原函数两次	I/O、副作用和费用翻倍	只调用一次，保存结果后再记录或返回
认为闭包会复制当前值	循环生成的函数全部读到最后一个配置	用工厂函数创建独立作用域

认为闭包会复制当前值	循环生成的函数全部读到最后一个配置	用工厂函数创建独立作用域
认为闭包状态按请求重建	计数和列表跨调用、跨请求共享	明确状态生命周期、隔离和并发策略
认为类型注解会校验模型 JSON	错类型仍可能到达函数体	在动态入口做运行时 schema 校验
重名时直接覆盖	导入顺序改变实际实现	写入前检查并明确失败

还有一个设计原则：普通函数或类已经能清楚表达的业务流程，不必为了"框架感"改成装饰器。装饰器适合稳定、横切、声明式的行为；复杂分支和大量隐式共享状态放在普通对象中通常更易维护。

16.10 运行本章示例

理解完前面的模型后，再用本地代码观察真实执行顺序。本章有四个文件：

文件	职责
`README.md`	独立教程和运行说明
`example.py`	七组可观察实验与断言
`tool_registry.py`	类型保真的同步 `@tool(...)` 和最小注册表
`registered_tools.py`	首次导入时执行注册的本地工具模块

仓库根目录的 `pyproject.toml` 要求 Python `>=3.14`。本章只使用标准库，不需要新增运行依赖。

在仓库根目录执行：

```
.venv/bin/python -B java_to_python/chapters/16_decorators/example.py
```

`-B` 只阻止生成 `__pycache__`，不改变程序语义。现有 `java_to_python/run_all.py` 只支持前 10 章，因此第 16 章应独立运行。

运行时重点观察：

1. 手工赋值和 `@` 写法都先进入包装层，再得到原结果；
2. 计时 wrapper 保留返回值、异常、原函数名、docstring 和对外签名；
3. 同一 wrapper 的计数累积，不同 wrapper 的计数彼此独立；

README | readme | README | README
wf7lz72502x6dg8fpjj0sff40000gp/T/crossnote-f-f695acc98bca1129d253380c8.html
相关... | eclp相关 | 配运运单26年需求... | 运单小秘值班安排 | 研发日常任务 | 运单运维 | 运单-资源大屏 | 20260618-应急预案 | 运单26年需求迭代... | 商家下单计数

运行时重点观察：

1. 手工赋值和 `@` 写法都先进入包装层，再得到原结果；
2. 计时 wrapper 保留返回值、异常、原函数名、docstring 和对外签名；
3. 同一 wrapper 的计数累积，不同 wrapper 的计数彼此独立；
4. 多层装饰器在定义期从内向外应用，调用期从外向内进入；
5. 忘记 `wraps` 与忘记 `return` 分别破坏元信息和返回值；
6. 导入工具模块后注册表已有三项，但业务调用事件仍为空；
7. 重名注册被拒绝，`@tool` 与 `@timed` 的顺序决定注册哪个对象。

最后应看到：

```
全部第 16 章实验与断言通过。
```

计时毫秒值每次都可能不同，不要把某次数字当成固定结果。

如需检查代码质量与静态类型，在仓库根目录执行：

```
.venv/bin/ruff check java_to_python/chapters/16_decorators
.venv/bin/ruff format ——check java_to_python/chapters/16_decorators
.venv/bin/pyright \
  java_to_python/chapters/16_decorators/example.py \
  java_to_python/chapters/16_decorators/tool_registry.py \
  java_to_python/chapters/16_decorators/registered_tools.py
```

16.11 递进练习

练习 1：手工展开装饰器

实现 `uppercase_result`，把无参函数返回的字符串转成大写。分别用普通赋值和 `@` 语法装饰两个函数，并写出各自的 `f = decorator(f)` 等价式。

验收：两个函数分别返回 `"AGENT"` 和 `"TOOL"`，并能说明装饰器应用和函数体调用各发生在何时。

练习 2：实现透明的 `@logged`

装饰 `divide(left: float, right: float, *, precision: int = 2) -> float`，调用前打印函数名。要求使用 `ParamSpec`、返回类型变量和 `wraps`，且不吞掉 `ZeroDivisionError`。

练习 2：实现透明的 `@logged`

装饰 `divide(left: float, right: float, *, precision: int = 2) -> float`，调用前打印函数名。要求使用 `ParamSpec`、返回类型变量和 `wraps`，且不吞掉 `ZeroDivisionError`。

验收：`inspect.signature(divide)` 仍展示原签名，正常结果和异常语义都不变。

练习 3：修复晚绑定并管理独立预算

为 `search`、`weather`、`database` 创建带独立标签和调用预算的 wrapper。每项预算从 2 开始，耗尽时抛出 `RuntimeError`。

验收：所有 wrapper 使用自己的标签和预算，并能解释哪里需要 `nonlocal`。

练习 4：比较注册与包装顺序

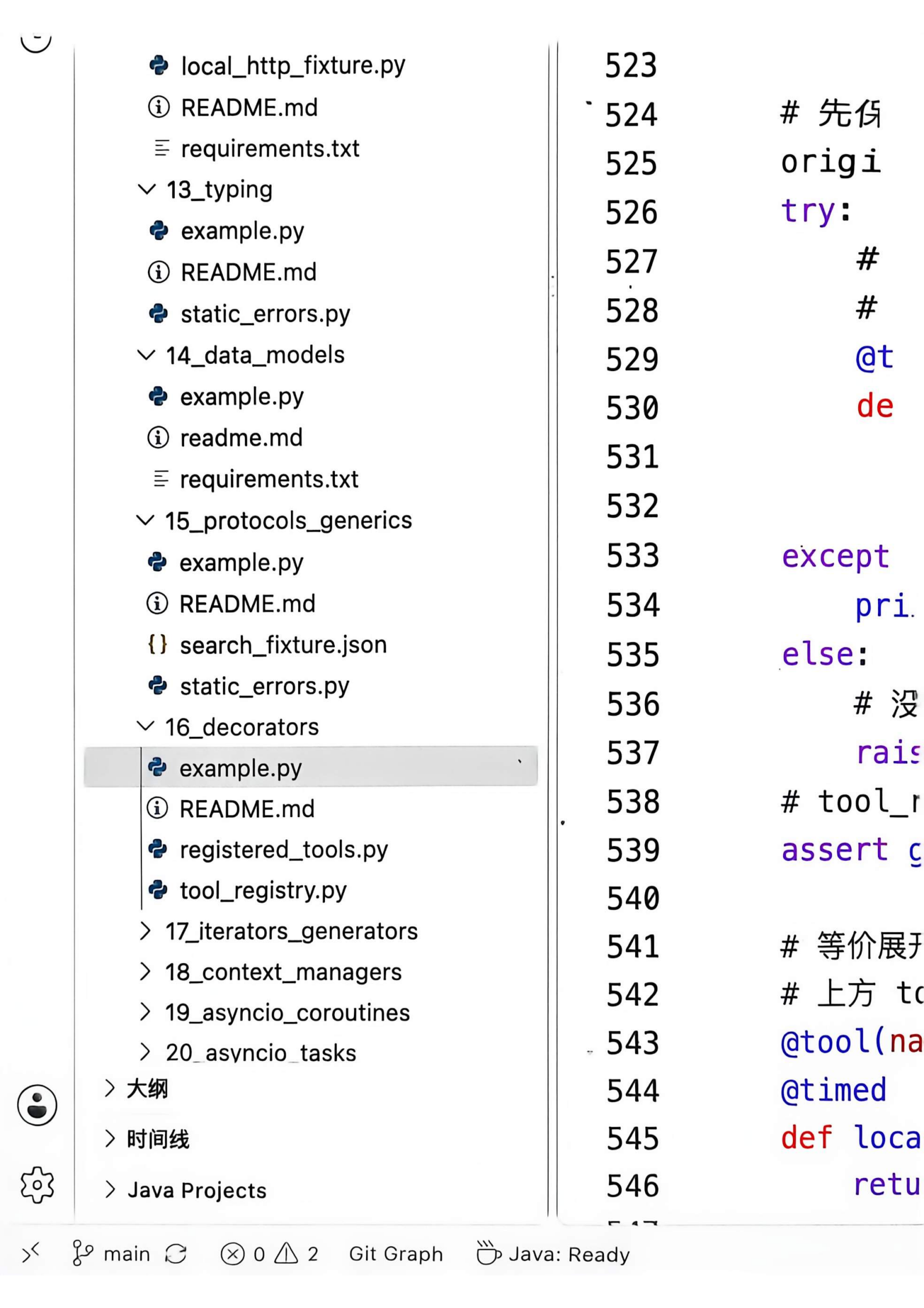
分别定义 `@tool @logged` 和 `@logged @tool` 两个函数，从公开名字和注册表各调用一次，用事件列表记录经过的层。

验收：先写出两组嵌套等价式，再解释注册表保存的是原函数还是 wrapper。

16.12 本章小结

- `@decorator` 的核心是 `name = decorator(original_function)`；
- 装饰器在执行 `def` 时应用，wrapper 在以后调用函数时执行；
- 闭包让 wrapper 记住原函数、配置和状态，重新绑定外层名字需要 `nonlocal`；
- 循环变量是晚绑定，想保存每轮值应创建独立作用域；
- 多层装饰器从内向外应用，最终调用从外向内进入；
- 透明 wrapper 必须正确转发参数、返回值和异常，并且只调用原函数一次；
- `wraps` 维护运行时元信息，`ParamSpec` 维护静态参数关系，两者不能互相替代；
- 注册型装饰器可以直接返回原函数，只有需要拦截每次调用时才创建 wrapper；
- 导入期可以完成轻量注册，但不应顺便执行业务 I/O；
- Python 3.14 应使用 `annotationlib` / `inspect` 的公开 API 读取延迟注解。

如果你能看到 `@tool(...)` 后立即说清“什么时候调用工厂、什么时候注册、什么时候才执行函数体，以及注册表里最终保存哪个对象”，就已经掌握了本章最重要的执行模型。


local_http_fixture.py
README.md
requirements.txt
13_typing
example.py
README.md
static_errors.py
14_data_models
example.py
readme.md
requirements.txt
15_protocols_generics
example.py
README.md
search_fixture.json
static_errors.py
16_decorators
example.py
README.md
registered_tools.py
tool_registry.py
17_iterators_generators
18_context_managers
19_asyncio_coroutines
20_asyncio_tasks
大纲
时间线
Java Projects
523
524 # 先保
525 origi
526 try:
527 #
528 #
529 @t
530 de
531
532
533 except
534 pri
535 else:
536 # 没
537 rais
538 # tool_r
539 assert g
540
541 # 等价展开
542 # 上方 to
543 @tool(na
544 @timed
545 def loca
546 retu
main
0 2
Git Graph
Java: Ready

```python
"""第 16 章：装饰器、闭包与同步工具注册。

所有实验都在本机同步完成。计时值会随机器变化；事件顺序、返回值、异常类型与
注册状态由断言验证，不依赖 sleep 或固定线程调度。
"""

# 建议按文件从上往下读：前半部分先“定义”装饰器和被装饰函数，后半部分的
# demonstrate_* 函数再“调用”它们并用断言验证现象。要始终区分两个时刻：
# 模块执行到 @decorator 时就会应用装饰器；被包装函数的函数体要等到显式调用才运行。

# 标准库导入按本章用途可以分成四类：
# - importlib 用于第二次导入实验，验证普通 import 与 import_module 都会复用模块缓存；
# - annotationlib、inspect 用于读取 Python 3.14 延迟注解与函数签名；
# - Callable、wraps 用于保存静态调用关系与运行时元信息；
# - perf_counter_ns 提供单调高分辨率计时，assert_type 则记录静态类型检查预期。
import importlib
from annotationlib import Format, ForwardRef, get_annotations
from collections.abc import Callable
from functools import wraps
from inspect import signature
from time import perf_counter_ns
from typing import assert_type

# 这些对象来自同目录的最小工具注册表。tool 负责注册；其余函数负责查询、动态调用和观察事件。
from tool_registry import (
    DuplicateToolError,
    get_tool,
    invoke_tool,
    list_tools,
    registration_events,
    tool,
)


# 这个小函数只统一七组实验的输出标题。`-> None` 表示它不向调用者返回有意义的结果。
def print_section(title: str) -> None:
    # f-string 会先计算花括号中的 title，开头的 \n 让不同实验之间空一行。
```

```
34
35  # 这个小函数只统一七组实验的输出标题。`-> None` 表示它不向调用者返回有意义的结果。
36  def print_section(title: str) -> None:
37      # f-string 会先计算花括号中的 title；开头的 \n 让不同实验之间空一行。
38      print(f"\n=== {title} ===")
39
40
41  # ------------------------------------------------------------------------
42  # 1. 从普通高阶函数推导 @decorator
43  # ------------------------------------------------------------------------
44  # 参数 function 是“函数对象”，不是函数调用的结果：`Callable[[], str]` 表示它不接收参数，
45  # 调用后返回 str。返回注解相同，表示 announce_call 最终也交回一个同契约的可调用对象。
46  def announce_call(function: Callable[[], str]) -> Callable[[], str]:
47      """最小无参装饰器：接收函数对象，返回另一个函数对象。"""
48
49      # 执行 announce_call 时只是创建 wrapper，并不进入 plain_message 的函数体。
50      # wrapper 引用了外层局部名字 function，因此与该绑定共同组成闭包；外层返回后仍能找到原函数。
51      def wrapper() -> str:
52          print(f"  包装层：准备调用 {function.__name__}")
53          # 这里的括号才真正调用原函数。return 把原结果继续交给 wrapper 的调用者。
54          return function()
55
56      # 返回函数对象时没有括号；若写成 wrapper()，会在装饰阶段过早执行它并返回字符串。
57      return wrapper
58
59
60  # 这是待装饰的原函数。执行 def 只创建函数对象，return 语句要等 plain_message() 才运行。
61  def plain_message() -> str:
62      return "普通形式完成"
63
64
65  # 右侧先把原函数对象交给 announce_call，返回的 wrapper 再绑定到新名字。
66  # 因而手工形式可清楚地看见核心模型：decorated = decorator(original)。
67  manually_decorated_message: Callable[[], str] = announce_call(plain_message)
68
69
```

```python

# @announce_call 是上面赋值模型的语法糖。解释器执行到这个 def 时先创建原函数对象，
# 随即调用 announce_call，再让 syntax_sugar_message 这个名字指向返回的 wrapper。
@announce_call
def syntax_sugar_message() -> str:
    return "@ 语法完成"


# ------------------------------------------------------------------------
# 2. 类型保真的同步计时装饰器
# ------------------------------------------------------------------------
# Python 3.14 的 `[**P, R]` 在本函数上声明两个类型参数：P 是整组参数规格，R 是返回类型。
# 输入、输出都写 Callable[P, R]，静态检查器便知道装饰前后的参数规则与结果类型应保持一致。
# 这些类型提示不会在运行时自动检查实参；真正的参数绑定仍由被调用函数完成。
def timed[**P, R](function: Callable[P, R]) -> Callable[P, R]:
    """记录同步调用耗时，并原样转发参数、返回值和异常。"""

    # timed(function) 执行时会定义 wrapper，并立即把原函数交给 wraps。
    # wraps 不负责计时；它只让返回的包装函数对日志、帮助系统和 inspect 更像原函数。
    @wraps(function)
    def wrapper(*args: P.args, **kwargs: P.kwargs) -> R:
        # 定义参数时，*args 把位置参数收集为 tuple，**kwargs 把关键字参数收集为 dict。
        # 每次调用 wrapper 都取一个新的起点；计时状态不会跨调用累计。
        started_ns: int = perf_counter_ns()
        try:
            # 调用位置上的 * 与 ** 做相反动作：把 tuple 和 dict 展开后交给原函数绑定。
            # return 很关键，否则原函数虽已执行，调用者却只能得到 wrapper 默认返回的 None。
            return function(*args, **kwargs)
        finally:
            # finally 在正常 return 和异常退出前都会运行。这里没有 except，所以不会吞掉或替换
            # 原异常；计时打印完后，正常结果或异常会沿原来的控制流继续向上传递。
            elapsed_ms: float = (perf_counter_ns() - started_ns) / 1_000_000
            # :.3f 把浮点毫秒数格式化为小数点后三位；具体数值随机器负载变化，不作断言。
            print(f"  [计时] {function.__name__}: {elapsed_ms:.3f} ms")

    # timed 必须返回 wrapper，外部函数名才会被重新绑定到带计时行为的新函数。
    return wrapper
```

```python
# 装饰器在模块加载、执行这个 def 后立刻应用；此时还不会构造问候语。
@timed
def build_greeting(prefix: str, name: str, /, *, punctuation: str = "!") -> str:
    """构造一条问候语。"""

    # 签名中的 `/` 表示它左侧的 prefix、name 只能按位置传入；单独的 `*` 表示其后的
    # punctuation 只能按关键字传入。wrapper 会原样转发，最终仍由这个签名执行约束。
    return f"{prefix}, {name}{punctuation}"


# 这个函数提供稳定的失败路径，用来证明 timed 的 finally 会执行，但 ValueError 不会被改写。
@timed
def require_positive(value: int) -> int:
    """用受控失败证明装饰器没有把异常改成其他结果。"""

    # 条件成立时 raise 创建并抛出 ValueError，当前函数不会继续执行下面的 return。
    if value <= 0:
        raise ValueError("value 必须大于 0")
    return value


# ---------------------------------------------------------------------------
# 3. 闭包状态、nonlocal 与晚绑定
# ---------------------------------------------------------------------------
# 每应用一次 @count_calls，都会重新调用此函数，创建一份 call_count 和一个新 wrapper。
# 所以同一 wrapper 的多次调用共享计数，不同被装饰函数之间却不会共享这一整数绑定。
def count_calls[**P, R](function: Callable[P, R]) -> Callable[P, R]:
    """每个 wrapper 拥有一个跨调用共享、但与其他 wrapper 独立的计数。"""

    # int 是不可变对象；后面的 += 不是修改原整数，而是计算新整数并重新绑定名字。
    call_count = 0

    @wraps(function)
    def wrapper(*args: P.args, **kwargs: P.kwargs) -> R:
        # nonlocal 指明这里使用最近一层外部函数中的 call_count，而不是创建 wrapper 的局部变量。
        # 若省略它，Python 会因下方存在赋值而把 call_count 判为局部名，读取旧值时就会报
```

```
134  def count_calls[**P, R](function: Callable[P, R]) -> Callable[P, R]:
139
140      @wraps(function)
141      def wrapper(*args: P.args, **kwargs: P.kwargs) -> R:
142          # nonlocal 指明这里使用最近一层外部函数中的 call_count，而不是创建 wrapper 的局部变量。
143          # 若省略它，Python 会因下方存在赋值而把 call_count 判为局部名，读取旧值时就会报
144          # UnboundLocalError。
145          nonlocal call_count
146          call_count += 1
147          print(f"  {function.__name__} 第 {call_count} 次调用")
148          return function(*args, **kwargs)
149
150      return wrapper
151
152
153  # 下面两次应用装饰器各自创建独立闭包。uppercase 调两次会显示 1、2，
154  # text_length 第一次调用仍显示自己的 1。
155  @count_calls
156  def uppercase(text: str) -> str:
157      return text.upper()
158
159
160  @count_calls
161  def text_length(text: str) -> int:
162      return len(text)
163
164
165  def make_bad_name_readers(names: list[str]) -> list[Callable[[], str]]:
166      """教学反例：所有闭包在调用时读取同一个循环变量 name。"""
167
168      # 列表元素是稍后才调用的无参函数，不是已经算好的字符串。
169      readers: list[Callable[[], str]] = []
170      # Python 的 for 循环不会像调用函数那样为每轮创建新局部作用域；所有轮次复用同一个 name。
171      for name in names:
172          # 每轮确实创建了不同的 read_name 函数对象，但它们闭包里引用的是同一个 name 绑定。
173          # 调用发生在循环结束后，届时该绑定保存的是 names 的最后一个元素。
```

```python

def make_bad_name_readers(names: list[str]) -> list[Callable[[], str]]:
    """教学反例：所有闭包在调用时读取同一个循环变量 name。"""

    # 列表元素是稍后才调用的无参函数，不是已经算好的字符串。
    readers: list[Callable[[], str]] = []
    # Python 的 for 循环不会像调用函数那样为每轮创建新局部作用域；所有轮次复用同一个 name。
    for name in names:
        # 每轮确实创建了不同的 read_name 函数对象，但它们闭包里引用的是同一个 name 绑定。
        # 调用发生在循环结束后，届时该绑定保存的是 names 的最后一个元素。
        def read_name() -> str:
            # 下一行的行尾抑制指令只忽略这个刻意保留的晚绑定告警，不应扩散到其他代码。
            return name  # noqa: B023

        # 这里只保存函数对象，没有 `()`，因此循环期间不会读取 name。
        readers.append(read_name)
    return readers


# 工厂函数用一次真实的函数调用制造一层新作用域；每次调用的参数 name 都是独立局部绑定。
def make_name_reader(name: str) -> Callable[[], str]:
    """每调用一次工厂就创建一个新作用域和独立绑定。"""

    # 返回的闭包只引用本次 make_name_reader 调用中的 name，所以之后循环变量再变化也不影响它。
    def read_name() -> str:
        return name

    return read_name


def make_good_name_readers(names: list[str]) -> list[Callable[[], str]]:
    # 推导式从左到右取出每个 name，并立即调用一次工厂；结果列表仍保存函数对象。
    # 等价于循环中反复执行 readers.append(make_name_reader(name))。
    return [make_name_reader(name) for name in names]


```

```python

# ---------------------------------------------------------------------------
# 4. 装饰器工厂与多层顺序
# ---------------------------------------------------------------------------
# layer 本身是“装饰器工厂”：先接收 label 和共享事件列表，再返回真正接收 function 的 decorate。
# 最外层返回类型可读作 `(Callable[P, R]) -> Callable[P, R]`，即一个类型保真的装饰器。
def layer[**P, R](
    label: str,
    events: list[str],
) -> Callable[[Callable[P, R]], Callable[P, R]]:
    """同时记录工厂求值、装饰器应用以及包装函数调用。"""

    # 这行位于工厂函数体内，所以求值 @layer(...) 表达式时立刻记录，早于装饰器应用和业务调用。
    events.append(f"求值装饰器表达式: {label}")

    # Python 创建完被装饰函数后，才把它传给 decorate；多层装饰器在此阶段从下往上应用。
    def decorate(function: Callable[P, R]) -> Callable[P, R]:
        events.append(f"应用装饰器: {label} -> {function.__name__}")

        # label、events 和 function 都来自外层作用域，wrapper 以后调用时仍能通过闭包访问它们。
        @wraps(function)
        def wrapper(*args: P.args, **kwargs: P.kwargs) -> R:
            events.append(f"进入包装层: {label}")
            try:
                return function(*args, **kwargs)
            finally:
                # 多层调用返回时由内向外退出；finally 也保证原函数抛异常时仍留下离开事件。
                events.append(f"离开包装层: {label}")

        # decorate 返回包装后的函数；它会成为再外一层装饰器的输入，或最终绑定到公开函数名。
        return wrapper

    # 工厂返回装饰器本身，不在这里应用它。
    return decorate


```

```python
# ------------------------------------------------------------------------
# 5. 两个可控反例
# ------------------------------------------------------------------------
# 这个反例正确转发了调用，却故意省略 @wraps(function)。运行结果仍对，运行时元信息却属于
# wrapper，说明“调用语义正确”和“内省元信息完整”是两件需要分别保证的事。
def without_wraps[**P, R](function: Callable[P, R]) -> Callable[P, R]:
    """教学反例：调用关系保留了，但运行时元信息没有复制。"""

    def wrapper(*args: P.args, **kwargs: P.kwargs) -> R:
        return function(*args, **kwargs)

    return wrapper


def drops_return[**P, R](function: Callable[P, R]) -> Callable[P, None]:
    """教学反例：原函数结果被调用了，却没有从 wrapper 返回。"""

    # 返回类型明确写成 None，是为了诚实描述这个错误 wrapper 的实际契约。
    def wrapper(*args: P.args, **kwargs: P.kwargs) -> None:
        # 原函数会执行并算出结果，但结果没有被 return；Python 到达函数末尾会隐式返回 None。
        function(*args, **kwargs)

    return wrapper


# 应用装饰器后，opaque_add 这个名字实际指向 without_wraps 内创建的 wrapper。
@without_wraps
def opaque_add(left: int, right: int = 1) -> int:
    """本应能被帮助系统看到的说明。"""

    return left + right


# broken_add 的原函数体会计算 left + right，但公开调用经过 wrapper 后收不到该整数。
@drops_return
def broken_add(left: int, right: int) -> int:
```

```
267
268 # broken_add 的原函数体会计算 left + right，但公开调用经过 wrapper 后收不到该整数。
269 @drops_return
270 def broken_add(left: int, right: int) -> int:
271     return left + right
272
273
274 def annotation_label(value: object) -> str:
275     """把普通类型、泛型别名和 ForwardRef 变成稳定的教学输出。"""
276
277     # 定义 render_card 时 ResultCard 尚不存在，FORWARDREF 模式会先得到这种占位对象。
278     if isinstance(value, ForwardRef):
279         # !r 使用 repr 风格，把字符串引号也放进稳定输出；__forward_arg__ 是其保存的类型名。
280         return f"ForwardRef({value.__forward_arg__!r})"
281     # 普通类对象统一只显示 __name__，避免默认字符串包含冗长的模块路径。
282     if isinstance(value, type):
283         return value.__name__
284     # list[str] 等泛型别名不属于 type 分支，用 str 得到适合教学展示的文本。
285     return str(value)
286
287
288 def demonstrate_equivalence() -> None:
289     print_section("1. @decorator 就是重新绑定函数名")
290
291     # 两个公开名字现在都指向 wrapper。调用它们时，先打印包装层信息，再通过闭包调用各自原函数。
292     manual_result: str = manually_decorated_message()
293     sugar_result: str = syntax_sugar_message()
294     # 两种写法的包装过程相同，差别只在“手工调用并赋值”还是使用 @ 语法表达。
295     print("  手工展开结果:", manual_result)
296     print("  @ 语法结果:", sugar_result)
297
298     # assert 条件为 False 时会抛 AssertionError，因此这里既校验返回值，也校验重新绑定后的对象身份。
299     assert manual_result == "普通形式完成"
300     assert sugar_result == "@ 语法完成"
301     # `is not` 比较是否为同一个对象，而不是内容是否相等；装饰后对象确实已换成 wrapper。
302     assert manually_decorated_message is not plain_message
303
```

```python

# 本组从静态类型、运行结果、内省元信息和异常路径四个角度检查 timed 是否“透明”。
def demonstrate_timing_and_metadata() -> None:
    print_section("2. 参数、返回值、异常与元信息都要保留")

    # 两个位置参数和一个关键字参数先被 wrapper 收集，再展开给 build_greeting 的原函数。
    # assert_type 在运行时原样返回问候语，同时要求 Pyright 推断装饰后的调用仍然得到 str。
    result: str = assert_type(build_greeting("你好", "Ada", punctuation="。"), str)
    # wraps 复制 __name__、__doc__ 并设置 __wrapped__，所以公开对象仍能展示业务函数信息。
    # signature 默认沿 __wrapped__ 链还原业务签名；follow_wrapped=False 才显示 wrapper 的
    # `*args: P.args, **kwargs: P.kwargs` 入口。
    print("  返回值:", result)
    print("  函数名:", build_greeting.__name__)
    print("  docstring:", build_greeting.__doc__)
    print("  对外签名:", signature(build_greeting))
    print("  wrapper 自身签名:", signature(build_greeting, follow_wrapped=False))
    print("  __wrapped__ 指向原函数:", build_greeting.__wrapped__ is not build_greeting)

    # 这些断言把“看起来正确”变成明确契约：结果、名字、位置/关键字规则都必须保持。
    assert result == "你好, Ada。"
    assert build_greeting.__name__ == "build_greeting"
    assert str(signature(build_greeting)) == (
        "(prefix: str, name: str, /, *, punctuation: str = '!') -> str"
    )
    # __wrapped__ 指向未包装的原函数。直接调用它会绕过 timed，因此这里不会输出计时信息。
    assert build_greeting.__wrapped__("你好", "Lin") == "你好, Lin!"

    # require_positive(0) 先进入计时 wrapper，再由原函数抛出 ValueError；finally 打印计时后，
    # 同一个异常继续来到这里的 except。这样能证明装饰器没有把失败悄悄改成 None。
    try:
        require_positive(0)
    except ValueError as error:
        # `as error` 把捕获到的异常对象绑定到局部变量；type(...).__name__ 取得异常类名。
        print("  异常原样传播:", type(error).__name__, str(error))
        assert str(error) == "value 必须大于 0"
    else:
        # try 没抛异常时才执行 else；走到这里反而说明实验失败，所以主动抛 AssertionError。
```

```
306  def demonstrate_timing_and_metadata() -> None:
339      else:
340          # try 没抛异常时才执行 else；走到这里反而说明实验失败，所以主动抛 AssertionError。
341          raise AssertionError("require_positive(0) 本应抛出 ValueError")
342
343
344  # 本组把两类“被闭包记住的值”放在一起比较：计数器需要跨调用共享，循环变量却不应共享。
345  def demonstrate_closures() -> None:
346      print_section("3. 闭包捕获与共享状态")
347
348      # uppercase 的两次调用进入同一个 wrapper，所以计数从 1 累加到 2；text_length 来自另一次
349      # @count_calls 应用，拥有另一份闭包状态，第一次调用从 1 开始。
350      print("  uppercase 返回:", uppercase("agent"))
351      print("  uppercase 返回:", uppercase("tool"))
352      print("  text_length 有自己的计数:", text_length("MCP"))
353
354      names: list[str] = ["search", "weather", "database"]
355      # 两个列表推导式都会逐个取出 reader、并立即调用 reader()，把返回的字符串收集成新列表。
356      # 坏版本的三个函数都在此刻读取循环结束后的同一 name；好版本各读自己工厂作用域中的 name。
357      bad_results: list[str] = [reader() for reader in make_bad_name_readers(names)]
358      good_results: list[str] = [reader() for reader in make_good_name_readers(names)]
359      print("  晚绑定反例:", bad_results)
360      print("  工厂创建独立绑定:", good_results)
361
362      # 第一条稳定复现晚绑定，第二条证明工厂按输入顺序保留了三个独立值。
363      assert bad_results == ["database", "database", "database"]
364      assert good_results == names
365
366
367  # 这个实验中的装饰器写在函数内部，因此每次调用本函数，都会重新定义 layered_work，
368  # 也会重新经历工厂求值、装饰器应用和 wrapper 调用三个阶段。
369  def demonstrate_decorator_order() -> None:
370      print_section("4. 多个装饰器：先猜顺序，再看事件")
371
372      # 同一个可变列表被两层装饰器和原函数闭包共同引用，用它记录确定的执行轨迹。
373      events: list[str] = []
374
```

```
367  # 这个实验中的装饰器写在函数内部，因此每次调用本函数，都会重新定义 layered_work，
368  # 也会重新经历工厂求值、装饰器应用和 wrapper 调用三个阶段。
369  def demonstrate_decorator_order() -> None:
370      print_section("4. 多个装饰器：先猜顺序，再看事件")
371
372      # 同一个可变列表被两层装饰器和原函数闭包共同引用，用它记录确定的执行轨迹。
373      events: list[str] = []
374
375      # 装饰器表达式先按源码从上到下求值：先 layer("外层")，再 layer("内层")。
376      # 原函数对象创建后，则按 `layer("外层")(layer("内层")(layered_work))` 从内向外应用。
377      @layer("外层", events)
378      @layer("内层", events)
379      def layered_work() -> str:
380          events.append("执行原函数体")
381          return "完成"
382
383      # 到 def 语句执行完，列表只包含“求值”和“应用”事件。copy 创建独立列表快照，避免后续
384      # 调用继续 append 时也改变 definition_events。
385      definition_events: list[str] = events.copy()
386      print("  定义阶段:", definition_events)
387      # 最终公开名字指向外层 wrapper：调用时先进入外层，再进入内层，最后才执行原函数体。
388      result: str = layered_work()
389      # 切片从定义期事件数量处取到末尾，产生只含调用期事件的新列表。
390      call_events: list[str] = events[len(definition_events) :]
391      print("  调用阶段:", call_events)
392
393      # 定义期是“表达式从上到下求值、装饰器从下到上应用”。
394      assert definition_events == [
395          "求值装饰器表达式：外层",
396          "求值装饰器表达式：内层",
397          "应用装饰器：内层 -> layered_work",
398          "应用装饰器：外层 -> layered_work",
399      ]
400      # 调用期从最外层向内进入，return 或异常退出时再从最内层向外经过各层 finally。
401      assert call_events == [
402          "进入包装层：外层",
```

```
369 def demonstrate_decorator_order() -> None:
400     # 调用期从最外层向内进入，return 或异常退出时再从最内层向外经过各层 finally。
401     assert call_events == [
402         "进入包装层：外层",
403         "进入包装层：内层",
404         "执行原函数体",
405         "离开包装层：内层",
406         "离开包装层：外层",
407     ]
408     assert result == "完成"
409
410
411 # 这里故意调用两个有缺陷但范围可控的装饰器，直接观察它们各自破坏了哪一部分契约。
412 def demonstrate_controlled_bugs() -> None:
413     print_section("5. 受控反例：忘记 wraps 或 return")
414
415     # opaque_add 的加法仍可用，但内省看到的是 wrapper 的名字、docstring 和宽泛参数签名。
416     print("  无 wraps 的名字:", opaque_add.__name__)
417     print("  无 wraps 的说明:", opaque_add.__doc__)
418     print("  无 wraps 的签名:", signature(opaque_add))
419     # broken_add 的原函数计算出 5，wrapper 却丢弃它并隐式返回 None。
420     print("  丢失 return 后:", broken_add(2, 3))
421
422     # 第一组断言证明“业务调用成功”不代表元信息正确；第二组锁定丢失 return 的后果。
423     assert opaque_add(2, right=3) == 5
424     assert opaque_add.__name__ == "wrapper"
425     assert not hasattr(opaque_add, "__wrapped__")
426     assert broken_add(2, 3) is None
427
428
429 # 本组把工具注册模块延迟到函数内部导入，便于在导入前后分别观察注册表和业务事件。
430 def demonstrate_import_time_registration() -> None:
431     print_section("6. 导入时注册，调用时才执行业务")
432
433     # 程序此前只导入了 tool_registry，它只定义注册能力，尚未导入任何带 @tool 的工具模块。
434     # list_tools 返回元信息元组；列表推导式这里只取每项的 name 用于展示。
```

```python
def demonstrate_import_time_registration() -> None:

    # 程序此前只导入了 tool_registry，它只定义注册能力，尚未导入任何带 @tool 的工具模块。
    # list_tools 返回元信息元组；列表推导式这里只取每项的 name 用于展示。
    print("  导入 registered_tools 前:", [item.name for item in list_tools()])
    assert list_tools() == ()

    # import 是可执行语句。第一次到达这里时，解释器从上到下执行 registered_tools 的顶层代码：
    # 每个 def 创建函数，紧邻的 @tool(...) 立即提取元信息并注册，但函数体中的业务代码不运行。
    import registered_tools

    # 导入完成后注册表已有三个工具；CALL_EVENTS 仍为空则证明搜索、汇总、渲染都未被调用。
    names_after_import: list[str] = [item.name for item in list_tools()]
    print("  导入后已注册:", names_after_import)
    print("  导入后业务调用事件:", registered_tools.CALL_EVENTS)
    print("  注册事件数:", len(registration_events()))
    assert names_after_import == ["search_docs", "summarize_results", "render_card"]
    assert registered_tools.CALL_EVENTS == []

    # 同一进程再次 import 命中 sys.modules 缓存，不会重跑模块顶层代码、装饰器或注册逻辑。
    # import_module 返回模块对象，因此可以用 is 验证它与第一次 import 绑定的是同一对象。
    same_module: ModuleType = importlib.import_module("registered_tools")
    print("  再次导入得到同一模块:", same_module is registered_tools)
    assert same_module is registered_tools
    assert len(registration_events()) == 3

    # 每轮 metadata 都是一个 ToolMetadata。内部字典推导式把注解映射中的键和值逐项转换成
    # 稳定展示文本；它创建新 dict，不会修改注册表保存的只读注解映射。
    for metadata in list_tools():
        annotation_text: dict[str, str] = {
            key: annotation_label(value) for key, value in metadata.annotations.items()
        }
        # 相邻 f-string 会在圆括号内自动拼接成一条字符串；!r 使用 repr，明确显示引号。
        print(
            f"  工具 {metadata.name!r}: python_name={metadata.python_name!r}, "
            f"signature={metadata.signature}, description={metadata.description!r}"
        )
```

```python
def demonstrate_import_time_registration() -> None:
    # 每轮 metadata 都是一个 ToolMetadata。内部字典推导式把注解映射中的键和值逐项转换成
    # 稳定展示文本；它创建新 dict，不会修改注册表保存的只读注解映射。
    for metadata in list_tools():
        annotation_text: dict[str, str] = {
            key: annotation_label(value) for key, value in metadata.annotations.items()
        }
        # 相邻 f-string 会在圆括号内自动拼接成一条字符串；!r 使用 repr，明确显示引号。
        print(
            f"  工具 {metadata.name!r}: python_name={metadata.python_name!r}, "
            f"signature={metadata.signature}, description={metadata.description!r}"
        )
        print("    注册时注解:", annotation_text)

    # @tool 的 decorate 只把原函数存进元信息并原样 return，没有创建 wrapper。
    search_metadata: ToolMetadata = get_tool("search_docs")
    print(
        "  @tool 返回原函数:",
        search_metadata.function is registered_tools.search_documents,
    )
    assert search_metadata.function is registered_tools.search_documents
    # 没有包装层也就没有 wraps 设置的 __wrapped__ 属性；这里不是“忘了 wraps”的缺陷。
    assert not hasattr(registered_tools.search_documents, "__wrapped__")

    # render_card 注册时 ResultCard 尚未定义，因此 Format.FORWARDREF 把 card 注解保存成占位对象。
    # 模块完成导入后类已经存在，再请求 Format.VALUE 就能把同一注解解析为真正的类对象。
    render_metadata: ToolMetadata = get_tool("render_card")
    # annotations 是“参数名 -> 注解值”的映射；用 ["card"] 取得 card 参数在注册时的注解。
    registered_card_annotation: object = render_metadata.annotations["card"]
    resolved_annotations: dict[str, AnnotationForm] = get_annotations(
        registered_tools.render_card,
        format=Format.VALUE,
    )
    print("  注册时 card 注解:", annotation_label(registered_card_annotation))
    print("  导入完成后 card 注解:", annotation_label(resolved_annotations["card"]))
    assert isinstance(registered_card_annotation, ForwardRef)
```

```
430  def demonstrate_import_time_registration() -> None:
491      assert isinstance(registered_card_annotation, ForwardRef)
492      # 类对象用 is 比较身份；这里只接受 registered_tools 中实际定义的 ResultCard 类。
493      assert resolved_annotations["card"] is registered_tools.ResultCard
494
495      # 到这里才真正调用三个工具函数，CALL_EVENTS 也将从空列表增长为三项。
496      # assert_type 同时记录静态预期，不会在运行时替我们检查 list[str] 或 str。
497      results: list[str] = assert_type(registered_tools.search_documents("装饰器", limit=2), list[str])
498      summary: str = assert_type(registered_tools.summarize_results(results, prefix="命中"), str)
499      card_text: str = assert_type(
500          registered_tools.render_card(registered_tools.ResultCard("装饰器", 9.5)),
501          str,
502      )
503      print("  调用后结果:", results, summary, card_text)
504      print("  调用后业务事件:", registered_tools.CALL_EVENTS)
505      assert results == ["Python 装饰器执行时机"]
506      assert summary == "命中: Python 装饰器执行时机"
507      assert card_text == "[9.5] 装饰器"
508      assert len(registered_tools.CALL_EVENTS) == 3
509
510      # 注册表能按运行时字符串找到并调用异构函数，但静态检查器无法从任意 str 推断该项的精确
511      # 参数和返回类型，所以统一入口只能诚实承诺 object；上面的具体函数直接调用仍保留精确类型。
512      dynamic_result: object = assert_type(
513          invoke_tool("summarize_results", results, prefix="动态调用"),
514          object,
515      )
516      print("  动态注册表调用:", dynamic_result, f"({type(dynamic_result).__name__})")
517      assert dynamic_result == "动态调用: Python 装饰器执行时机"
518
519
520  # 最后一组先验证失败注册不会破坏旧状态，再展示“只注册”和“包装执行”两种装饰器如何组合。
521  def demonstrate_registration_conflict_and_composition() -> None:
522      print_section("7. 重名拒绝与注册/包装组合")
523
524      # 先保存当前已注册函数对象，稍后用身份比较确认重名失败后它没有被覆盖。
525      original = get_tool("search_docs").function
```

```python

# 最后一组先验证失败注册不会破坏旧状态，再展示“只注册”和“包装执行”两种装饰器如何组合。
def demonstrate_registration_conflict_and_composition() -> None:
    print_section("7. 重名拒绝与注册/包装组合")

    # 先保存当前已注册函数对象，稍后用身份比较确认重名失败后它没有被覆盖。
    original = get_tool("search_docs").function
    try:
        # 执行到这个局部 def 时，tool(...) 先生成 decorate，随后 decorate 尝试注册新函数并抛出
        # DuplicateToolError。conflicting_search 的函数体不会运行，公开名字的绑定也不会完成。
        @tool(name="search_docs", description="这个定义故意制造重名。")
        def conflicting_search(query: str) -> str:
            return query

    except DuplicateToolError as error:
        print("  重名注册已拒绝:", str(error))
    else:
        # 没有异常才会进入 else；这意味着注册器意外接受了冲突，应让实验明确失败。
        raise AssertionError("重名工具本应被拒绝")
    # tool_registry 会先检查重名再写入，因此失败后 search_docs 仍指向原对象。
    assert get_tool("search_docs").function is original

    # 等价展开是 `local_sum = tool(...)(timed(local_sum))`：先由下方 timed 创建 wrapper，
    # 上方 tool 再读取这个 wrapper 的元信息、注册它并原样返回。wraps 让注册器仍看到业务签名。
    @tool(name="timed_local_sum", description="展示注册装饰器与执行包装器的组合。")
    @timed
    def local_sum(left: int, right: int) -> int:
        return left + right

    # 公开名字与注册表保存的是同一个 timed wrapper；该 wrapper 的 __wrapped__ 才是原 local_sum。
    metadata: ToolMetadata = get_tool("timed_local_sum")
    print("  公开函数就是已注册函数:", local_sum is metadata.function)
    print("  但 timed 创建了 wrapper:", local_sum is not local_sum.__wrapped__)
    print("  组合后签名:", metadata.signature)
    assert local_sum is metadata.function
    assert local_sum is not local_sum.__wrapped__
    # 因为注册的是 wrapper，从公开名字或注册表调用都会经过 timed 并打印耗时
```

```python
def demonstrate_registration_conflict_and_composition() -> None:

    # 公开名字与注册表保存的是同一个 timed wrapper；该 wrapper 的 __wrapped__ 才是原 local_sum。
    metadata: ToolMetadata = get_tool("timed_local_sum")
    print("  公开函数就是已注册函数:", local_sum is metadata.function)
    print("  但 timed 创建了 wrapper:", local_sum is not local_sum.__wrapped__)
    print("  组合后签名:", metadata.signature)
    assert local_sum is metadata.function
    assert local_sum is not local_sum.__wrapped__
    # 因为注册的是 wrapper，从公开名字或注册表调用都会经过 timed 并打印耗时。
    assert local_sum(2, 3) == 5


# main 只负责编排实验；函数定义顺序不等于执行顺序，真正调用从这里按列出的次序发生。
def main() -> None:
    demonstrate_equivalence()
    demonstrate_timing_and_metadata()
    demonstrate_closures()
    demonstrate_decorator_order()
    demonstrate_controlled_bugs()
    demonstrate_import_time_registration()
    demonstrate_registration_conflict_and_composition()
    print("\n全部第 16 章实验与断言通过。")


# 模块被直接当脚本运行时 __name__ 等于 "__main__"，于是调用入口；若被别的模块导入，
# 上面的定义期代码和模块级装饰器仍会执行，但不会自动跑七组 demonstrate_* 实验。
if __name__ == "__main__":
    main()

```

```python
"""用于证明“模块导入时注册，函数调用时才执行业务”的本地工具模块。"""

from dataclasses import dataclass

from tool_registry import tool

CALL_EVENTS: list[str] = []
LOCAL_DOCUMENTS = (
    "Python 装饰器执行时机",
    "闭包与 nonlocal",
    "用 ParamSpec 保留函数签名",
)


@tool(name="search_docs", description="在确定性的本地标题中搜索关键字。")
def search_documents(query: str, /, *, limit: int = 2) -> list[str]:
    """返回最多 limit 个包含 query 的本地标题。"""

    # 只有真正调用工具时才记录业务事件。模块导入和装饰器注册不会走到这里。
    CALL_EVENTS.append(f"调用 search_documents(query={query!r}, limit={limit})")
    if not query.strip():
        raise ValueError("query 不能为空")
    if limit < 1:
        raise ValueError("limit 必须大于 0")
    return [title for title in LOCAL_DOCUMENTS if query.lower() in title.lower()][:limit]


@tool()
def summarize_results(items: list[str], *, prefix: str = "结果") -> str:
    """把本地搜索结果拼成一行；工具名和说明回退到函数名与 docstring。"""

    CALL_EVENTS.append(f"调用 summarize_results(count={len(items)})")
    return f"{prefix}: {'、'.join(items) if items else ' (空) '}"


@tool(name="render_card", description="渲染一个在函数定义之后才声明的结果对象。")
def render_card(card: ResultCard) -> str:
```

```python
@tool()
def summarize_results(items: list[str], *, prefix: str = "结果") -> str:
    """把本地搜索结果拼成一行；工具名和说明回退到函数名与 docstring。"""

    CALL_EVENTS.append(f"调用 summarize_results(count={len(items)})")
    return f"{prefix}: {'、'.join(items) if items else ' (空) '}"


@tool(name="render_card", description="渲染一个在函数定义之后才声明的结果对象。")
def render_card(card: ResultCard) -> str:
    """演示 Python 3.14 延迟注解和未解析前向引用。"""

    CALL_EVENTS.append(f"调用 render_card(title={card.title!r})")
    return f"[{card.score:.1f}] {card.title}"


@dataclass(frozen=True, slots=True)
class ResultCard:
    """故意放在 render_card 后面；Python 3.14 定义函数时不会立即求值该注解。"""

    title: str
    score: float

```

```python
"""第 16 章使用的最小同步工具注册表。

这个模块只做三件事：构造元信息、拒绝重名、保存同步函数。它不发送网络请求，
也不尝试验证任意外部参数；参数边界验证属于后续 Agent 运行时的职责。
"""

from annotationlib import Format, get_annotations
from collections.abc import Callable, Mapping
from dataclasses import dataclass
from inspect import Signature, getdoc, signature
from types import MappingProxyType

type AnySyncFunction = Callable[..., object]


class DuplicateToolError(ValueError):
    """新工具试图占用已有名称时抛出，避免悄悄覆盖。"""


@dataclass(frozen=True, slots=True)
class ToolMetadata:
    """注册时保存的、只读的工具描述。"""

    name: str
    python_name: str
    description: str
    signature: Signature
    annotations: Mapping[str, object]
    function: AnySyncFunction


_REGISTRY: dict[str, ToolMetadata] = {}
_REGISTRATION_EVENTS: list[str] = []


def tool[**P, R](
    *
```

```python
def tool[**P, R](
    *,
    name: str | None = None,
    description: str | None = None,
) -> Callable[[Callable[P, R]], Callable[P, R]]:
    """返回同步工具装饰器，同时保留原函数的参数与返回类型关系。

    这个装饰器只注册，不包装执行，所以 decorate 最后直接返回 function。
    `Callable[P, R] -> Callable[P, R]` 告诉类型检查器：装饰前后的调用契约相同。
    """

    def decorate(function: Callable[P, R]) -> Callable[P, R]:
        resolved_name: str = function.__name__ if name is None else name.strip()
        resolved_description: str | None = getdoc(function) if description is None else description.strip()

        if not resolved_name:
            raise ValueError("工具名称不能为空")
        if not resolved_description:
            raise ValueError(f"工具 {resolved_name!r} 缺少 description 或 docstring")
        if resolved_name in _REGISTRY:
            # 先检查再写入，失败时不会留下半注册状态。
            raise DuplicateToolError(f"工具名称已注册: {resolved_name}")

        # Python 3.14 的注解默认延迟求值。FORWARDREF 能保留尚未定义的名称，
        # 不要求注册时所有前向引用都已经可以解析。
        annotations: dict[str, Any | ForwardRef] = get_annotations(function, format=Format.FORWARDREF)
        metadata = ToolMetadata(
            name=resolved_name,
            python_name=function.__name__,
            description=resolved_description,
            # STRING 格式适合面向人的工具描述；不依赖 __annotations__ 的内部形态。
            signature=signature(function, annotation_format=Format.STRING),
            annotations=MappingProxyType(dict(annotations)),
            function=function,
        )
        _REGISTRY[resolved_name] = metadata

```

```
36  def tool[**P, R](
47      def decorate(function: Callable[P, R]) -> Callable[P, R]:
72
73          event: str = f"注册 {resolved_name!r} <- {function.__name__}{metadata.signature}"
74          _REGISTRATION_EVENTS.append(event)
75          print(f"  [导入期] {event}")
76
77          # 注册与包装是两件事。这里不创建.wrapper，因此函数身份也保持不变。
78          return function
79
80      return decorate
81
82
83  def get_tool(name: str) -> ToolMetadata:
84      """按名称取得元信息；未知名称通过 LookupError 明确失败。"""
85
86      try:
87          return _REGISTRY[name]
88      except KeyError:
89          raise LookupError(f"未注册工具: {name}") from None
90
91
92  def list_tools() -> tuple[ToolMetadata, ...]:
93      """按注册顺序返回不可变快照，调用者不能借此修改内部字典。"""
94
95      return tuple(_REGISTRY.values())
96
97
98  def registration_events() -> tuple[str, ...]:
99      """返回注册事件快照，供导入时机实验观察。"""
100
101     return tuple(_REGISTRATION_EVENTS)
102
103
104 def invoke_tool(name: str, /, *args: object, **kwargs: object) -> object:
105     """演示按运行时字符串动态调用时，静态签名关系会在边界处被擦除。
106
```

```
 92  def list_tools() -> tuple[ToolMetadata, ...]:
 95      return tuple(_REGISTRY.values())
 96
 97
 98  def registration_events() -> tuple[str, ...]:
 99      """返回注册事件快照，供导入时机实验观察。"""
100
101      return tuple(_REGISTRATION_EVENTS)
102
103
104  def invoke_tool(name: str, /, *args: object, **kwargs: object) -> object:
105      """演示按运行时字符串动态调用时，静态签名关系会在边界处被擦除。
106
107      单个被装饰函数仍保留精确类型；但普通 dict 无法让类型检查器根据任意字符串，
108      自动关联每个异构工具各自不同的参数和返回类型。因此这个动态入口返回 object。
109      """
110
111      return get_tool(name).function(*args, **kwargs)
112
```